

Variable-Structure Automaton Inference from Interaction Sequences

Engin Durmaz

May 14, 2026

Abstract

This report documents work toward inferring a Variable-Structure Automaton (VSA) from a unique dataset of state–action–feedback tuples generated in a P-model environment. We summarize the mathematical background for empirical counts and conditional estimates, describe the implemented methodology for Tasks 1 and 2 (estimation and optimal-action identification) with references to the Python codebase, and outline planned experiments for the remaining assignment tasks.

1 Introduction

The term project asks us to reconstruct, from observed sequences only, the state–action policy, transition dynamics, penalty structure, and norm of behavior of an unknown automaton with five states $\{\phi_1, \dots, \phi_5\}$, three actions $\{\alpha_1, \alpha_2, \alpha_3\}$, and binary environment feedback $\beta \in \{0, 1\}$ (P-model). Each dataset row is a tuple (sequence_id, step, $\Phi_{\text{src}}(n)$, $\Phi_{\text{dst}}(n)$, $\alpha(n)$, $\beta(n)$). This document aligns notation with the course assignment and with the implementation under `../proj_python/`.

2 Background

2.1 Empirical count $N(\phi_i, \alpha_k)$

The number of times action α_k was taken while the source state was ϕ_i is

$$N(\phi_i, \alpha_k) = \#\{n : \Phi_{\text{src}}(n) = \phi_i \wedge \alpha(n) = \alpha_k\}. \quad (1)$$

In the CSV, $\Phi_{\text{src}}(n)$ is the column `phi_src` and $\alpha(n)$ is `alpha`.

2.2 Policy estimate $P(\alpha_k \mid \phi_i)$

Empirical policy probabilities normalize the counts in each state:

$$P(\alpha_k \mid \phi_i) = \frac{N(\phi_i, \alpha_k)}{\sum_{k'} N(\phi_i, \alpha_{k'})} = \frac{\text{times } \alpha_k \text{ taken in } \phi_i}{\text{times } \phi_i \text{ was visited}}. \quad (2)$$

The denominator is the total number of actions chosen from state ϕ_i .

2.3 Row totals `row_totals_phi`

In code, for each state ϕ_i ,

$$\text{row_totals_phi}[\phi_i] = \sum_k N(\phi_i, \alpha_k), \quad (3)$$

i.e. the sum of the $N(\phi_i, \cdot)$ row in the count matrix `N_phi_alpha`.

2.4 Transition and penalty counts (brief)

Similarly, $N(\phi_i, \alpha_k, \phi_j)$ counts transitions $\phi_i \rightarrow \phi_j$ under action α_k , and $N(\alpha_k, \beta)$ counts (action, feedback) pairs. These yield transition probabilities $P(\phi_j \mid \phi_i, \alpha_k)$ and penalty probabilities $c_k = P(\beta = 1 \mid \alpha_k)$ by row-wise normalization of the appropriate count tables.

3 Methodology

All runnable code lives in `../proj_python/. main.py` orchestrates tasks; each task module can be run standalone (writing outputs under `outputs/`) or imported (returning structured results while still printing to the console).

3.1 Task 1 — Estimation

We load the headerless UTF-8 CSV, build `N_phi_alpha` by grouping on `phi_src` and `alpha`, build per-action transition count matrices `N_phi_alpha_phi`, build `N_alpha_beta`, then derive policy, transition, and penalty (c_k). The following excerpt is the core link between Equations (1)–(3) and the implementation.

```
# count by grouping how many action alpha taken in state phi
N_phi_alpha = (
    df.groupby(["phi_src", "alpha"]).size()
    .unstack(fill_value=0)
    .reindex(index=states, columns=actions, fill_value=0)
)
N_phi_alpha.index.name = "phi_src"
N_phi_alpha.columns.name = "alpha"

# count by grouping how many transitions from state phi_src to state phi_dst for
# action a
N_phi_alpha_phi: Dict[str, pd.DataFrame] = {}
for a in actions:
    tab = (
        df[df["alpha"] == a]
        .groupby(["phi_src", "phi_dst"]).size()
        .unstack(fill_value=0)
        .reindex(index=states, columns=states, fill_value=0)
    )
    tab.index.name = "phi_src"
    tab.columns.name = "phi_dst"
    N_phi_alpha_phi[a] = tab

# count by grouping how many action alpha valuated with beta
N_alpha_beta = (
    df.groupby(["alpha", "beta"]).size()
    .unstack(fill_value=0)
    .reindex(index=actions, columns=betas, fill_value=0)
)
N_alpha_beta.index.name = "alpha"
N_alpha_beta.columns.name = "beta"

# Conditional probabilities -----
# N_phi_alpha : how many action alpha taken in state phi
# N_phi_alpha_phi : how many transitions from state phi_src to state phi_dst for
#                   action a
# N_alpha_beta : how many action alpha valuated with beta
```

```

row_totals_phi = N_phi_alpha.sum(axis=1)
policy = N_phi_alpha.div(row_totals_phi.replace(0, np.nan), axis=0).fillna(0.0)

transition: Dict[str, pd.DataFrame] = {}
for a in actions:
    N = N_phi_alpha_phi[a]
    T = N.div(N.sum(axis=1).replace(0, np.nan), axis=0).fillna(0.0)
    transition[a] = T

total_per_action = N_alpha_beta.sum(axis=1).replace(0, np.nan)
penalty = (N_alpha_beta[1] / total_per_action).fillna(0.0)
penalty.name = "c_k"

```

Consistency checks verify that policy rows and transition rows (where data exist) sum to unity, and that c_k matches the empirical mean of β per action. See `estimation.py` lines 171–193.

3.2 Task 2 — Optimal action identification

Under the P-model, the optimal action minimizes the penalty probability:

$$\alpha^* \in \arg \min_k c_k, \quad c_k = P(\beta = 1 \mid \alpha_k). \quad (4)$$

The implementation reads c_k from the in-memory `EstimationResult` when called from `main.py`, or from `penalty.csv` after a standalone Task 1 run. Ties at the minimum are resolved deterministically (first label in sorted order).

```

if estimation is not None:
    penalty = estimation.penalty.astype(float).copy()
    penalty.index.name = "alpha"
    penalty.name = "c_k"
    source = "estimation-result"
else:
    in_dir = Path(input_dir) if input_dir is not None else DEFAULT_INPUT
    penalty = _load_penalty_from_files(in_dir)
    source = str((in_dir / PENALTY_FILE).resolve())

if penalty.empty:
    raise ValueError("Penalty vector is empty; nothing to optimize over.")

c_min = float(penalty.min())
ties: List[str] = [str(a) for a in penalty.index[np.isclose(penalty.values, c_min)]]
alpha_star = ties[0] # deterministic pick: first action tied at the minimum

ranking = (
    penalty.sort_values(ascending=True)
        .to_frame()
        .assign(rank=lambda d: np.arange(1, len(d) + 1, dtype=int))
)

```

3.3 Tasks 3–6 (planned)

- **Task 3 (ε -optimality):** $\varepsilon = 1 - \min_i P(\alpha^* \mid \phi_i)$ using the policy matrix from Task 1 and α^* from Task 2.
- **Task 4 (simulation):** sample synthetic trajectories from the learned policy and transition model; compare marginal distributions to the dataset.

- **Task 5 (perturbation):** randomly remove 20% of transitions and re-estimate c_k , policy, and ε .
- **Task 6 (model critique):** estimate c_k separately per source state and compare across states to assess the P-model assumption.

These will extend `main.py` with new modules mirroring the dual-run convention used for Tasks 1–2.

4 Experiments

Data. The provided file `../Engin-vsa_dataset_full.csv` (one row per step; Greek state/action labels) is the default input for `estimation.py`.

Reproducibility.

1. Task 1 only (writes CSVs and `report.txt`): `cd ../proj_python && python estimation.py`
2. Task 2 after Task 1 files exist: `cd ../proj_python && python optimal_action_identification.py`
3. End-to-end in memory (no intermediate files required): `cd ../proj_python && python main.py`

Planned experiments for the final report will include the dataset fingerprint checksum, full numeric tables for Tasks 1–2, and (once implemented) figures and tables for simulation match, perturbation robustness, and per-state penalty comparison.

5 Conclusion

We fixed notation for empirical counts and policy normalization, and showed how `N_phi_alpha`, `row_totals_phi`, and `policy` in `estimation.py` implement Equations (1)–(3). Task 2 selects α^* by minimizing c_k from the same estimation pipeline. The remaining tasks complete the assignment’s data-driven story: optimality diagnostics, generative validation, robustness under missing transitions, and critique of the P-model assumption.